

To cast the choices values as valid or false, we'll use conditional binding in conjunction with a double exclamation (!!). In other words, if the options alternative isn't identified, it won't be made at all. The dynamic Vue component can now act uniformly in all fields.

```
// form-template.vue

<component :is="InputBox"
  :options="!!field.options.choices? field.options.choices: false"
/>
```

Using component

Copyright Attribution:
Medium

Binding the Various Form Inputs

When creating forms in Vue.js, you may want to take advantage of the user's comments to improve the user interface. For eg, an online newsletter. To sign up, you'll usually be prompted to enter your password. The name you got would then feature in the email's greeting after you subscribe to the newsletter. Data binding is the term for this process.

The v-model directive in Vue.js allows you to build two-way data bindings on form inputs, text, textareas, and select fields.

The v-model directive selects the appropriate method of updating the entity based on the input form. By binding the input text entity and the value binded to a variable allocated, it provides two-way data binding. For various types of input objects, the v-model directive uses different properties and emits different events. In general, three forms of binding are used in form input binding:

- Binding to a Textarea: We use text and textarea to attach the value property and input case in this binding.
- Select and Radio Binding: Checkboxes and radiobuttons are used to attach the verified property and alter events in this binding.
- Binding Modifiers: In Form input binding instances, we may also use modifiers like lazy, trim, number, and so on.

What is the concept of two-way data binding?

The v-model directive is one way to bind data in Vue.js. As this model is attached to an input, it tracks the input and updates the assigned prototype if the model changes. If the prototype is modified, it also changes the data model. This is referred to as "two-way data binding."